



Embedded Systems

Introduction

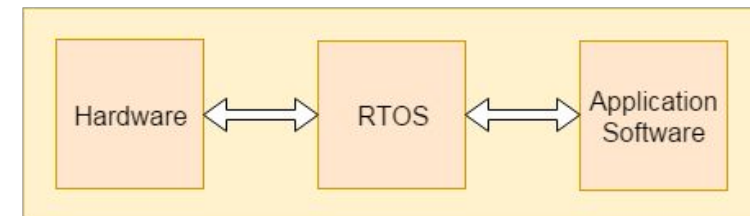
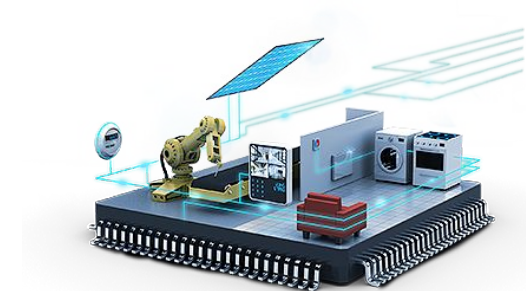
Embedded Systems

❖ System

- An arrangement where all its assembled components work according to some specific defined rules

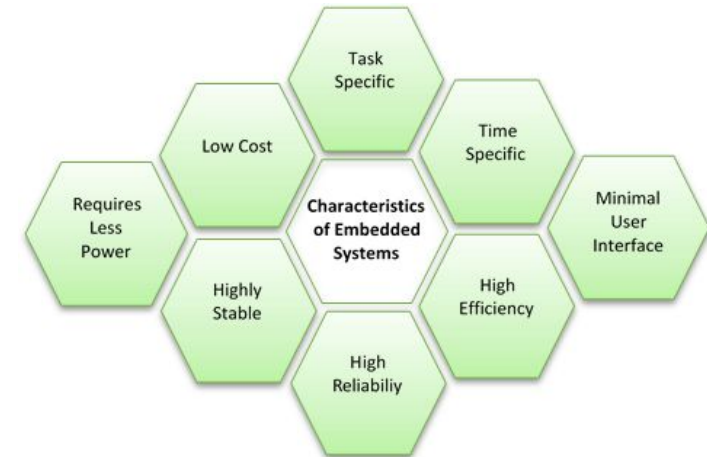
❖ Embedded System (A computer system embedded into a larger system)

- A microcontroller or microprocessor based control system which is designed to perform a specific task
- Usually has three components
 - Hardware
 - Application Software
 - Real Time Operating system (RTOS)
 - Not always required. E.g. a simple application
- Examples: Medical equipments, automobiles, industry, routers, robotics and etc.



Embedded Systems

- ❖ Dedicated and designed for a specific task
 - Usually performs a specific operation repeatedly
 - Unlike general purpose computers which run general purpose softwares like browsers, games and etc.
- ❖ Real time computation ability
 - Continually reacts to changes in the system's environment and computes certain results in real time without any delay
- ❖ Highly available, stable and reliable
 - The less failure probability of a system means more reliability
 - A Failure can possibly lead a self-driving car to an accident



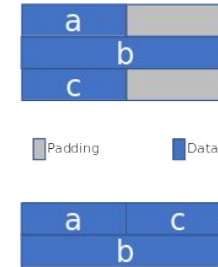
Embedded Systems

- ❖ Highly available, stable, reliable and deterministic
 - The less failure probability of a system means more reliability
 - A Failure can possibly lead a self-driving car to an accident
- ❖ Flexible: a built-in debugging opportunity that allows for remote maintenance. e.g. a spacecraft
- ❖ Portable: Using the same embedded software in various environments
 - Using abstraction layers between the business logic and the low-level system interfaces
- ❖ Connected: must be connected to peripheral devices and communicate with external world
 - Analog and digital sensors
 - Analog and digital actuators
 - Communication interfaces
 - E.g. UART, SPI, CAN and etc.

Embedded Systems

❖ Tightly constrained

- Low cost, low power, small size
- Limited memory and processing power
 - Data types matter. Members order and alignment matters.
 - Program size and runtime size matter. Every byte matters.
- More safety, single threaded, high performance and etc.



```
struct Bad_Align
{
    uint8_t a;
    uint16_t b;
    uint8_t c;
};

struct Good_Align
{
    uint8_t a;
    uint8_t c;
    uint16_t b;
};
```

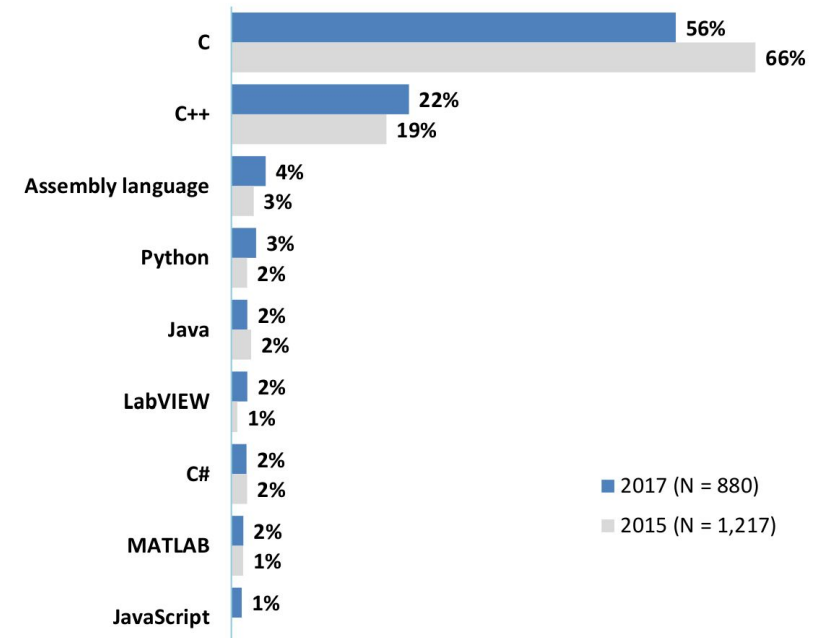
❖ Microprocessors based: must be microprocessor or microcontroller based

❖ Non-volatile memories

- Flash memories to embed applications. Secondary memory is not necessary required.
- EEPROM or emulated EEPROM to store dynamic calibration and configuration values

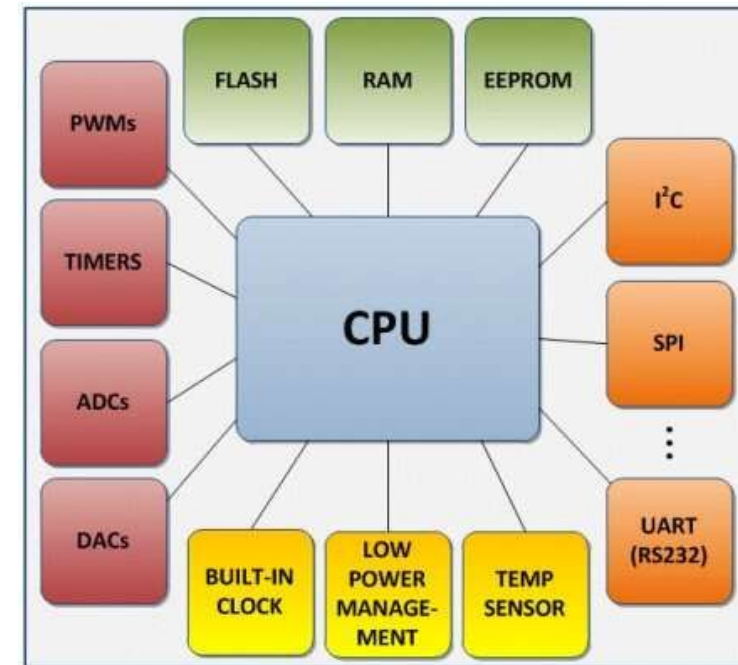
Embedded Systems

- ❖ Fault-Tolerant: Shall survive in the presence of faults
 - All environments are assumed to be noisy
- ❖ Advantages
 - Easily Customizable
 - Low power consumption
 - Low cost and
 - Enhanced performance
- ❖ Disadvantages
 - Complex and requires more time. Skilled engineers are required
- ❖ Programming languages: C, C++, Embedded Java, Python and etc.
 - C and C++ are the most used. Not all features of C/C++ are used.



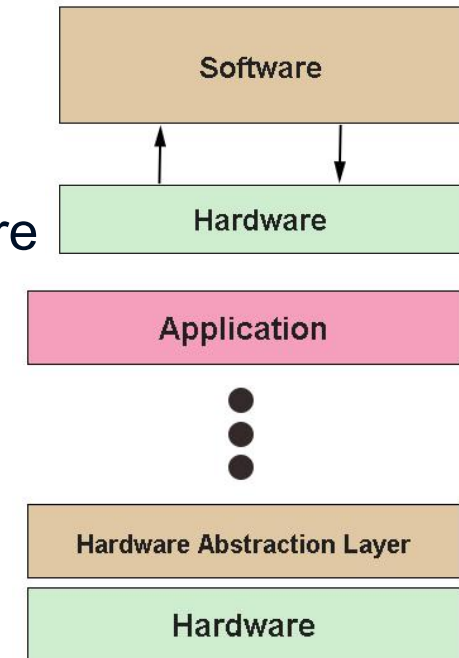
Embedded Systems (Basic Structure)

- ❖ Sensor: Used for sensing the change in environment conditions and generate electrical signals
- ❖ A-D Converter: Is a device to convert an analog input signal into its equivalent digital signal
- ❖ Memory: Used to store data. Embedded systems have volatile and non-volatile memories
- ❖ Processor: It processes the data to measure the output and store it in the memory
- ❖ D-A Converter: It converts the digital data fed by the processor to analog data
- ❖ Actuator: A mechanical or electrical device which converts signals to equivalent physical actions
- ❖ Communication interfaces
- ❖ And etc.



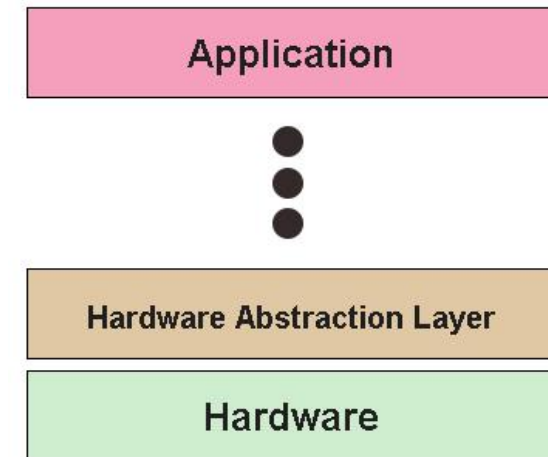
Computer System

- ❖ It is a machine designed to retrieve, process and store numerical data
- ❖ It can be a general-purpose or an embedded computer system
 - But the basic principles of operation and the underlying architectures are the same
- ❖ It is composed of hardware and software
- ❖ The hardware executes the software
- ❖ The software controls the operation and functionality of the hardware
- ❖ The architecture of the softwares can be different
 - E.g. AUTOSAR, Linux based general-purpose systems and etc.
 - But the first layer in the software architectures is the **Hardware Abstraction Layer (HAL)**



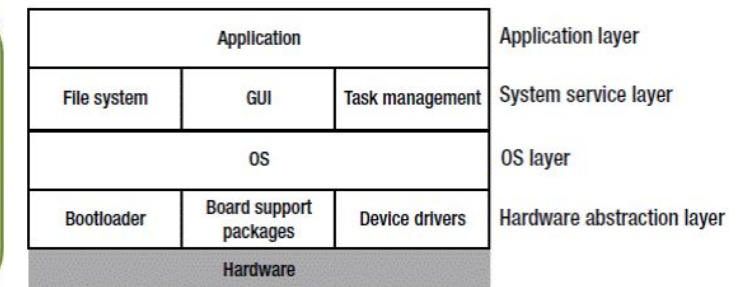
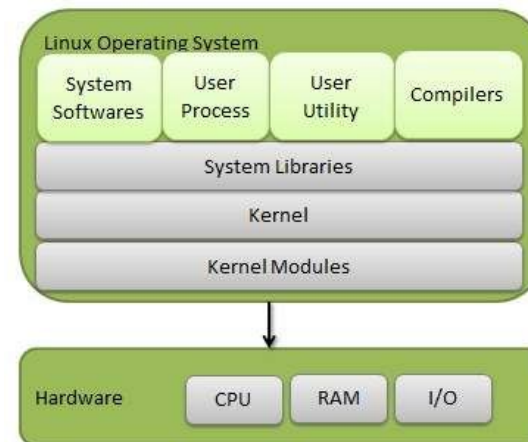
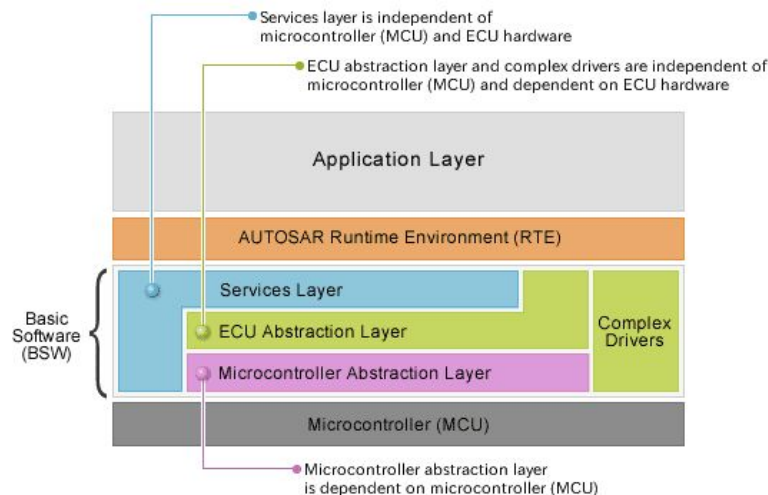
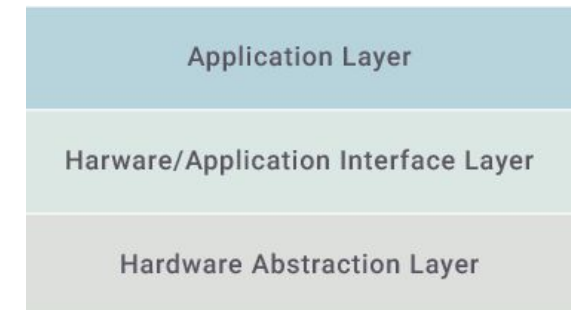
Hardware Abstraction Layer

- ❖ **HAL** is the first layer in the software architecture of a computer system
- ❖ It makes softwares hardware independent
- ❖ It increases portability and testability of softwares
- ❖ It is the only layer which directly interacts with the hardware
- ❖ It can for example contain
 - Board Support Package of a microcontroller
 - Firmware, device drivers, BIOS, bootloader and etc.
- ❖ A device **driver** is a software that controls and interacts directly with the device and provides some functionalities for the upper layers by some interfaces.



Computer System Architecture

- ❖ Between the HAL and the Application layer, generally there is an interface layer
 - Provides services and abstractions for applications
 - Depending on applications, it has its own architecture
 - In complex systems an OS is used in this layer
 - E.g. AUTOSAR, Linux and etc.



Typical Software Architecture of an embedded system

Basic Computer System Architecture

❖ Three components connected to a bus system

➤ Processor

- The computing and controlling part of a computer system which processes the data

➤ Memory

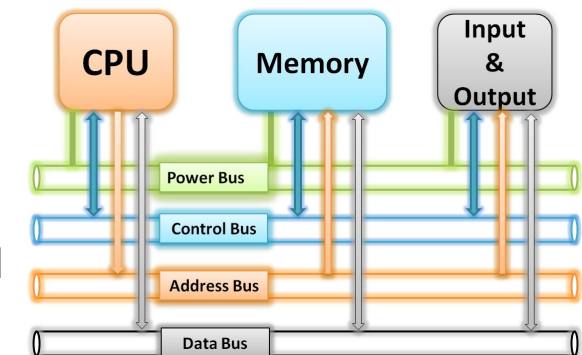
- Is used by the processor to retrieve and store data

➤ I/O Devices

- Are used by the processor to communicate with the external world

❖ Processor

- A processor is an electronic device capable of processing data based on an instruction set
- There are two major instruction set architecture (ISA) for processors
 - Complex Instruction Set Computer (**CISC**) processors; e.g. Intel x86 processors
 - Reduced Instruction Set Computer (**RISC**) processors; e.g. ARM processors like Cortex-M4



Computer System Architecture

Microprocessor vs. Microcontroller	
Microprocessor	Microcontroller
A computational unit. Nothing else	A computer system; Microprocessor, Memory, Storage and I/O and etc.
General purpose, non specific task	Specialized and used in embedded systems
High clock speed (> 1.5GHz)	Moderate clock speed (40-600 MHz)
No memory	On chip memory (Both RAM & ROM)
External memory controller	In circuit memory controller
No HW interfaces	Embedded HW interfaces
Requires a lot of support components	Can work almost alone
Runs an OS	Usually no OS
64 bit architecture	8, 16 or 32 bit architecture

Computer System Architecture

Microprocessor vs. Microcontroller

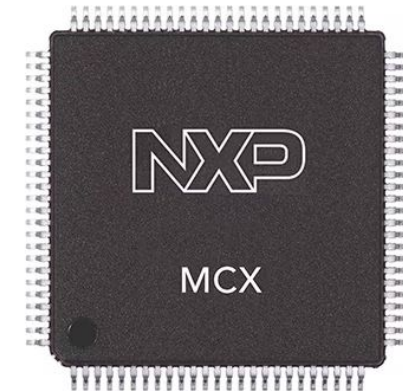
Microprocessor - Ex. Intel Core i9-13900K

- 24 cores, 32 threads
- No I/O
- Memory Controller - < 128Gb
- Up to 5.8 GHz
- CISC - Complex Instruction Set Computing
- 64 bit architecture



Microcontroller - Ex. NXP MCXN236

- ARM Cortex-M33 core, FPU, MPU, DSP and etc.
- < 100 I/O pins, inkl. CAN, SPI, I2C, UART, PWM, etc.
- 352KB RAM, 1MB Flash, 256KB ROM
- 150 MHz
- RISC - Reduced Instruction Set Computing
- 32 bit architecture



Embedded Systems

❖ Microcontrollers

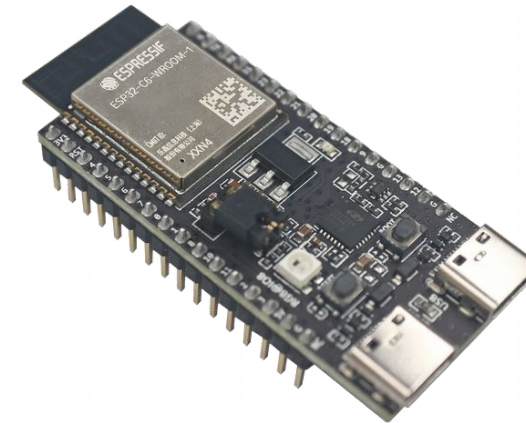
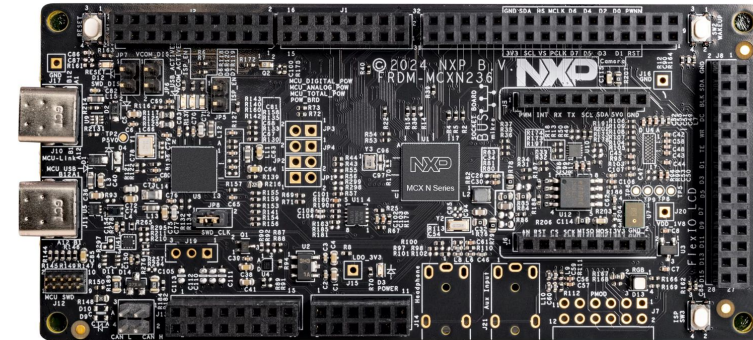
➤ Freedom Development Board for MCX N23x MCUs

- [FRDM-MCXN236](#)
- [MCX-N23X](#)
- [Zephyr Project](#)
- [Zephyr API](#)

➤ ESP32-C6-DevKitC-1 Development Board

- [ESP32-C6-DevKitC-1](#)
- [Documentation and Resources](#)
- [ESP-IDF Programming Guide](#)
- [API Reference](#)

❖ Tools: Visual Studio Code, PlatformIO IDE and MCUXPresso for VS Code



Embedded Systems

❖ Some useful links

- [Electronic Systems](#)
- [Embedded Systems Tutorials](#)
- [5 Steps to Getting Started with Embedded Programing](#)
- [Embedded Systems Tutorial: History, Types, Advantages, Examples](#)
- [Embedded system](#)
- [Introduction to the C Programming Language for Embedded Applications](#)
- [Introduction to Embedded Systems](#)
- [What is an Embedded System?](#)
- [5 Differences between Embedded Software Engineer and Software Developer](#)
- [C Programming Exercises, Practice, Solution](#)